Course: Computer Vision

Course Code : ITA0506

Slot: A

Faculty: Jeni Gracia R J

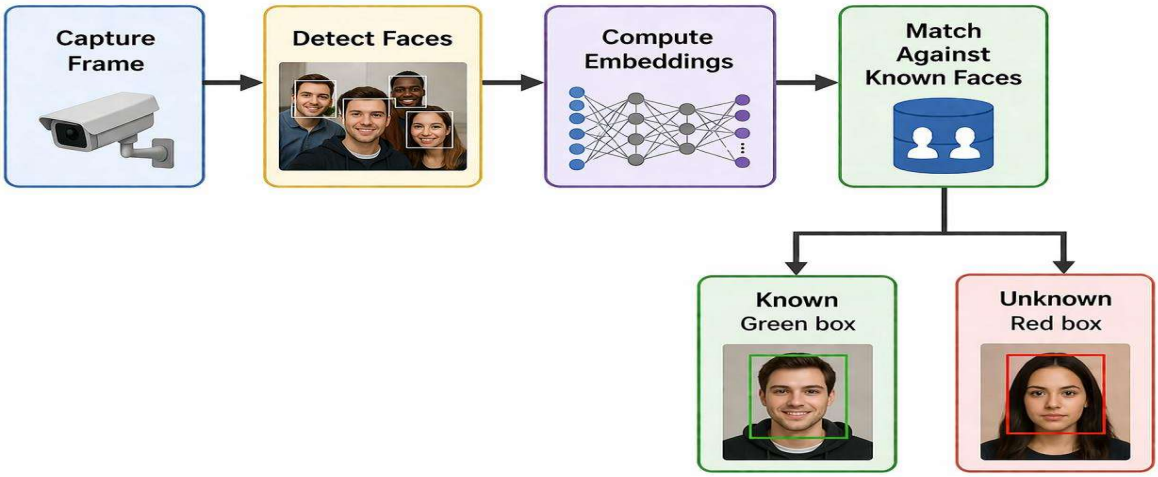
Submitted By

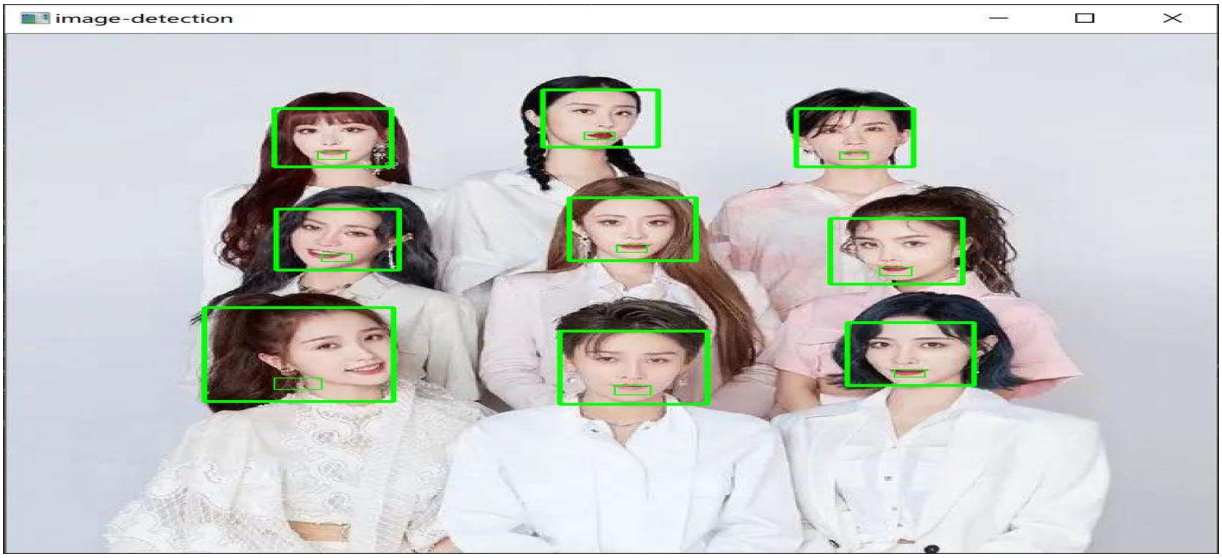
Y Sravan Kumar (192472289)

CO5_ASSESSMENTS 2

PSEUDOCODE WRITING TASK

1. REAL-TIME FACE DETECTION AND RECOGNITION

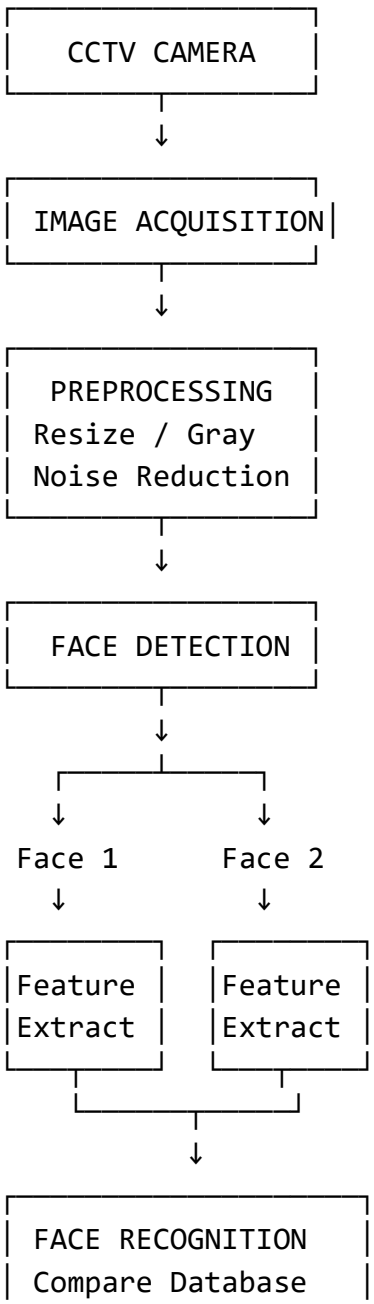


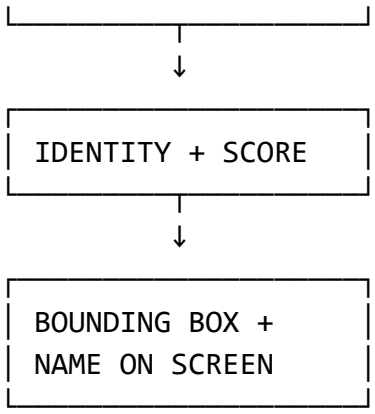


Aim

To design a real-time OpenCV-based face detection and recognition system that captures video frames, detects multiple faces, recognizes registered persons, and displays bounding boxes with identity labels.

System Architecture





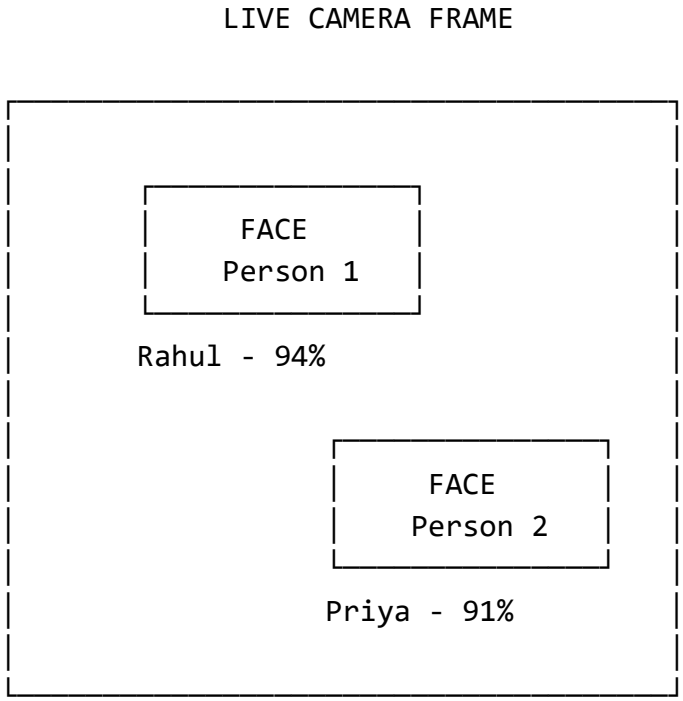
Detailed Pseudocode

```
BEGIN

    Load OpenCV library

    Load trained face recognition model

    Load registered face dataset

    Initialize camera

    IF camera is not available THEN
        Display "Camera Not Found"
        STOP
    END IF

    WHILE camera is running

        Capture a frame

        IF frame is not captured THEN
            BREAK
        END IF

        Resize the frame

        Preprocess the frame
```

```
Detect all faces in the frame

FOR each detected face

    Extract face region

    Resize face to required size

    Convert to grayscale if required

    Normalize face image

    Extract facial features

    Compare features
    with registered dataset

    Calculate confidence score

    IF confidence is greater than threshold THEN
        Assign registered person's name
    ELSE
        Assign "Unknown"
    END IF

    Draw bounding box around face

    Display person's name

    Display confidence score

END FOR

Display processed frame

IF user presses 'q' or ESC THEN
    BREAK
END IF
```

```
    END WHILE

    Release camera

    Close all OpenCV windows

END
```

Picture/Working Example



Important Steps

1. Image Acquisition

The camera continuously captures frames.

```
Camera → Frame 1 → Frame 2 → Frame 3 → Frame 4 → ...
```

2. Preprocessing

The image is improved using:

- Resizing
- Grayscale conversion
- Noise reduction
- Normalization

3. Face Detection

The system identifies multiple faces.

```
Frame
 ↓
Face Detector
 ↓
Face 1 + Face 2 + Face 3
```

4. Feature Extraction

Important facial information is converted into numerical features.

```
Face Image
     ↓
Feature Extraction
     ↓
Feature Vector
```

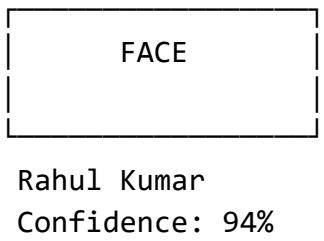
5. Recognition

The feature vector is compared with the registered database.

```
Input Face
    ↓
Feature Extraction
    ↓
Database Comparison
    ↓
Best Match
    ↓
Person Name
```

6. Visualization

The system displays a bounding box and identity.



Efficient Loop Control

To maintain real-time performance:

- Resize frames.
- Use lightweight face detectors.
- Process only the required region.
- Avoid unnecessary calculations.
- Use tracking when appropriate.
- Process selected frames if required.
- Stop when `q` or `ESC` is pressed.

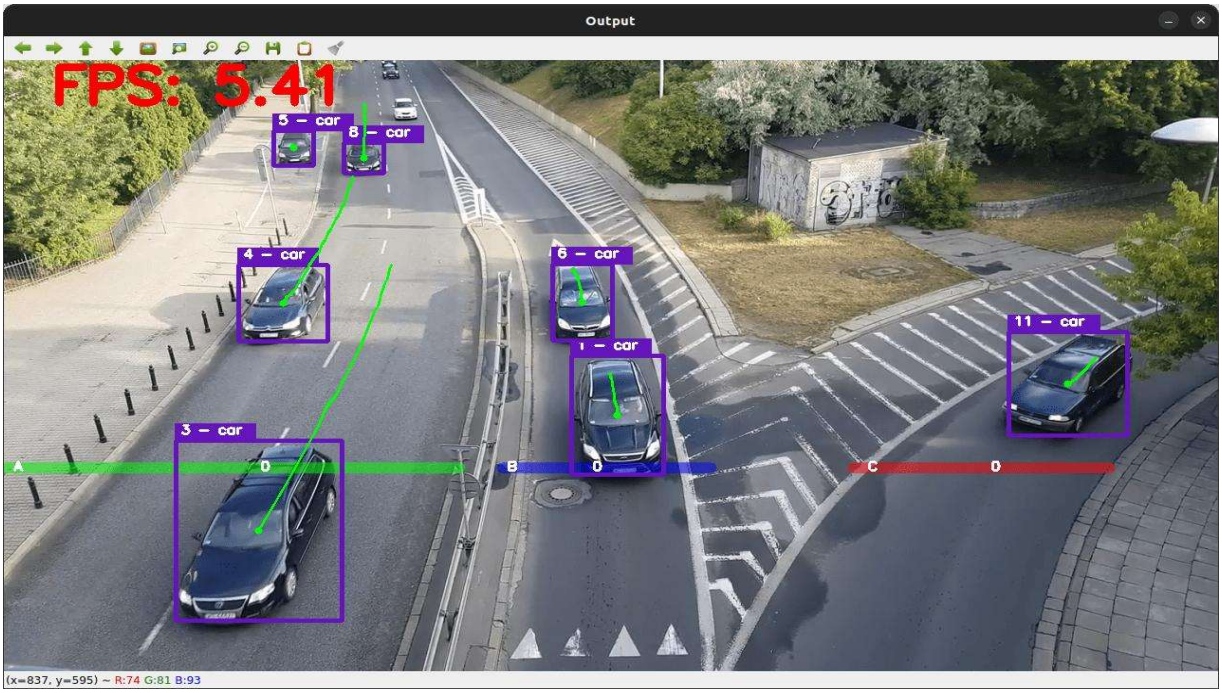
Expected Output

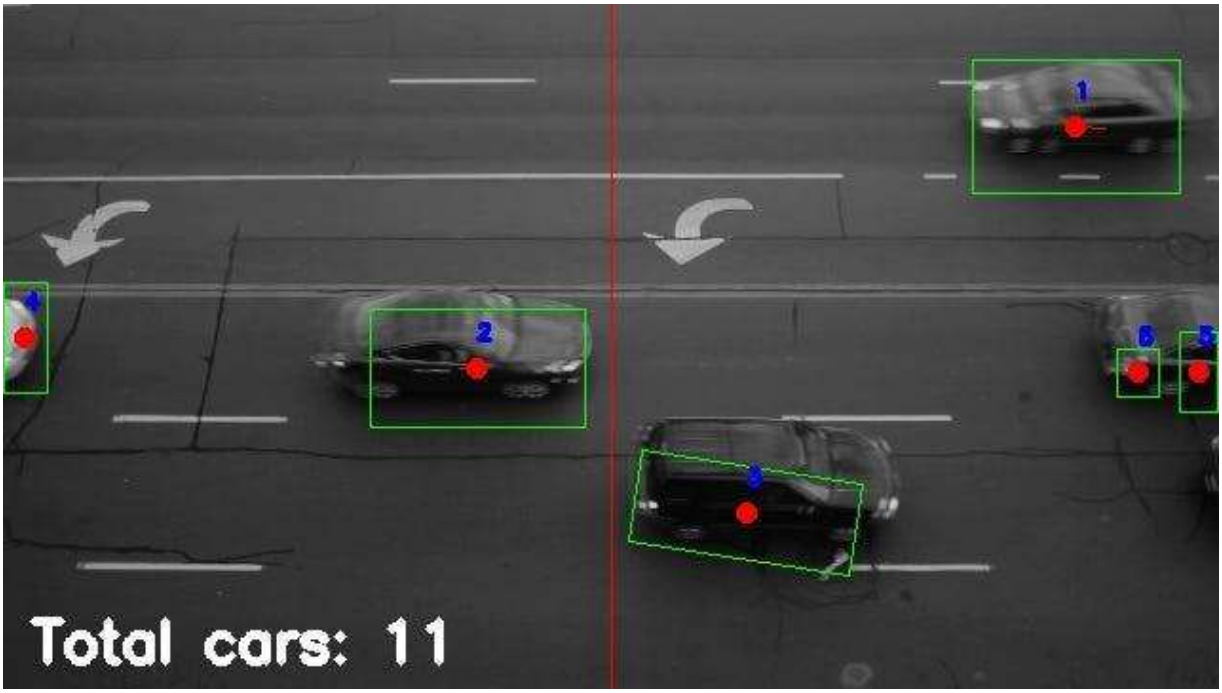
```
Face Detected
Name       : Rahul Kumar
Confidence : 94%

Face Detected
Name       : Priya
Confidence : 91%

Face Detected
Name       : Unknown
Confidence : Low
```

Result

A complete pseudocode design for a **real-time multi-face detection and recognition system** was developed. The system detects faces, extracts features, recognizes registered persons, and displays their identities using OpenCV.

2. VEHICLE DETECTION, TRACKING AND COUNTING



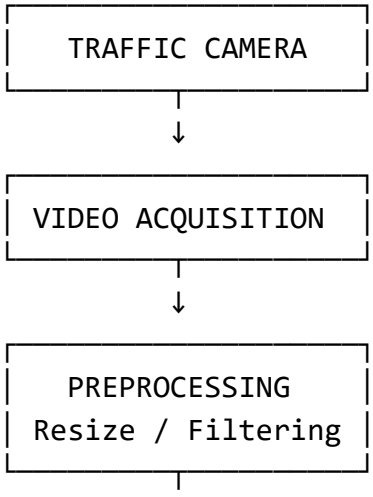


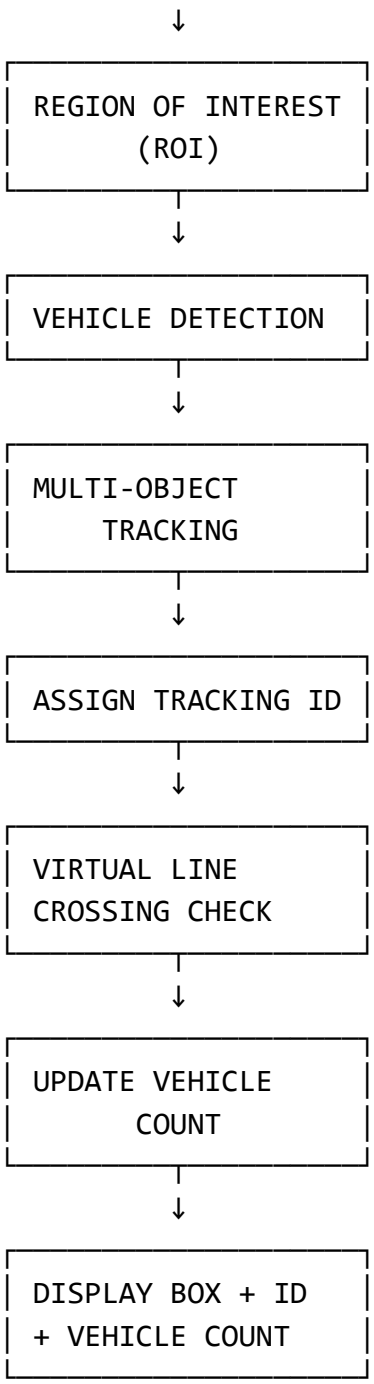
6

Aim

To design an OpenCV-based vehicle detection, tracking, and counting system that detects vehicles in a predefined region, tracks them across frames, counts vehicles crossing a virtual line, and displays tracking IDs.

System Architecture





Detailed Pseudocode

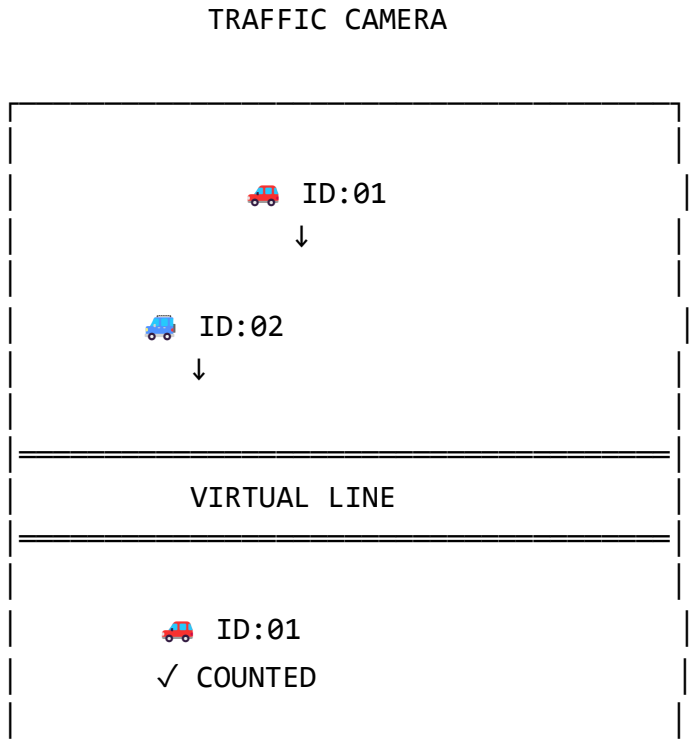
```
BEGIN

    Load OpenCV library
```

```
Load vehicle detection model

Initialize vehicle tracker

Set vehicle_count = 0

Create empty set counted_vehicles

Define Region of Interest

Define virtual counting line

Open live traffic camera

IF camera is not available THEN
    Display "Camera Not Found"
    STOP
END IF

WHILE video is running

    Capture a frame

    IF frame is not captured THEN
        BREAK
    END IF

    Resize frame

    Apply preprocessing

    Extract Region of Interest

    Detect vehicles inside ROI

    FOR each detected vehicle

        Get bounding box
```

```
    Calculate center point

END FOR

Send detected vehicles
to tracking algorithm

FOR each tracked vehicle

    Assign unique tracking ID

    Get current center point

    Get previous center point

    Draw bounding box

    Display tracking ID

    Check whether vehicle
    crossed virtual line

    IF vehicle crossed line THEN

        IF tracking ID is not
           in counted_vehicles THEN

            vehicle_count =
            vehicle_count + 1

            Add tracking ID
            to counted_vehicles

        END IF

    END IF

END FOR

Draw ROI
```

```
        Draw virtual counting line

        Display vehicle count

        Display processed frame

        IF user presses 'q' or ESC THEN
            BREAK
        END IF

    END WHILE

    Release video

    Close all OpenCV windows

END
```

Picture/Working Example



```
| Vehicle Count : 15                    |
└──────────────────────────────────────┘
```

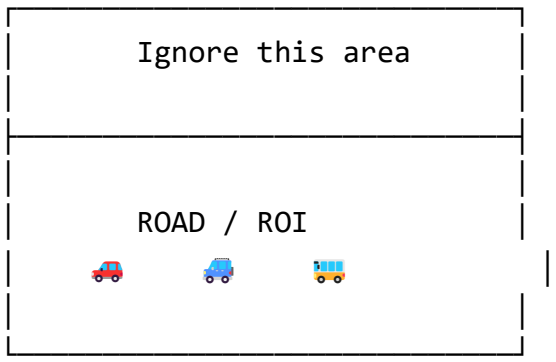
Important Steps

1. Video Acquisition

```
Traffic Camera
      ↓
Live Video
      ↓
Frames
```

2. Region of Interest

Only the road area is processed.



This reduces processing time.

3. Vehicle Detection

The system detects vehicles such as:

- 🚗 Car
- 🚌 Bus
- 🚚 Truck
- 🏍️ Motorcycle

4. Tracking

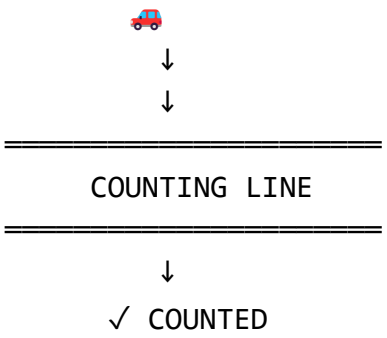
Each vehicle receives a unique ID.

```
Car       → ID 01
Bus       → ID 02
Truck     → ID 03
Car       → ID 04
```

The ID remains associated with the same vehicle across consecutive frames.

5. Line Crossing

A virtual line is placed on the road.



6. Avoiding Double Counting

Suppose vehicle ID 01 crosses the line.

```
ID 01 → Crossed
   ↓
Check counted list
   ↓
Not present
   ↓
Count = Count + 1
   ↓
Add ID 01
```

If ID 01 appears again:

```
ID 01 → Already counted
              ↓
        Do NOT count again
```

This is the **most important part** of the vehicle-counting algorithm.

Multiple Vehicle Tracking

```
Frame 1:

🚗 ID:01        🚙 ID:02        🚌 ID:03
       ↓               ↓               ↓

Frame 2:

🚗 ID:01        🚙 ID:02        🚌 ID:03
       ↓               ↓               ↓

Frame 3:

🚗 ID:01        🚙 ID:02        🚌 ID:03
```

The tracker maintains the identity of each vehicle.

Efficient Loop Control

For near real-time performance:

- Resize video frames.
- Process only ROI.
- Use a lightweight detection model.
- Use an efficient tracking algorithm.
- Avoid unnecessary detection.
- Process selected frames if needed.
- Stop using `q` or `ESC`.

Expected Output

```
Vehicle ID : 01
Type       : Car
Status     : Crossed

Vehicle ID : 02
Type       : Bus
Status     : Moving

Vehicle ID : 03
Type       : Truck
Status     : Crossed

---------------------------
Total Vehicle Count : 18
---------------------------
```

Result

A complete pseudocode design for a **real-time vehicle detection, tracking, and counting system** was developed. The system uses ROI processing, vehicle detection, tracking IDs, virtual-line crossing, and duplicate-count prevention to accurately count multiple vehicles.